PART 08 — ARCHITECTURE DECISION RECORDS (ADRS)

An **Architecture Decision Record (ADR)** documents a significant software architecture choice made along with its context, options considered, and the trade-offs accepted.

ADR-001: Decimal Arithmetic via `Decimal.js` and PostgreSQL `NUMERIC(19,4)`

- **Status:** Accepted & Enforced
- **Context & Non-Technical Explanation:**

Standard computer processors perform math using binary floating-point numbers (IEEE 754 standard). In everyday programming, this causes subtle inaccuracies:

$$\text{In JavaScript: } 0.1 + 0.2 = 0.30000000000000004$$

In consumer video games or social media apps, a rounding discrepancy of 0.00000000000000004 is invisible. However, in an institutional accounting engine processing millions of journal lines, these microscopic rounding errors accumulate into fractional pennies. When the general ledger is summed at year-end, Total Debits and Total Credits will fail to balance by 1 or 2 pence, triggering statutory audit failures.

- **Decision:**

All financial amounts in TypeScript are strictly handled via `Decimal.js` wrapped inside `MoneyService`. In PostgreSQL, all money fields are typed as `NUMERIC(19,4)`. Floating-point numbers (`number` in TypeScript, `float/double` in SQL) are strictly banned from financial data flows.

- **Consequences & Trade-offs:**
- *Positive:* Guarantees zero arithmetic drift. Debits and credits balance to the exact fourth decimal place.
- *Negative:* Arithmetic operations require explicit method calls (`a.plus(b)` instead of `a + b`), with a minor compute overhead that is negligible for transactional workloads.

ADR-002: Multi-Tenant Boundary Enforcement via Composite Foreign Keys

- **Status:** Accepted & Enforced
- **Context & Non-Technical Explanation:**

In multi-tenant cloud software, multiple companies share the same database tables. Traditionally, engineers protect data purely in software code by adding `where: { organization_id: current_org_id }` to every query. If an engineer accidentally omits this line during a midnight bug fix, Client A can see or edit Client B's invoices.

- **Decision:**

Enforce composite foreign keys at the PostgreSQL database schema level:

`sql

ALTER TABLE "journal_lines"

ADD CONSTRAINT "journal_lines_organization_id_contact_id_fkey"

FOREIGN KEY ("organization_id", "contact_id")

REFERENCES "contacts"("organization_id", "id")

ON DELETE RESTRICT ON UPDATE CASCADE;

- **Consequences & Trade-offs:**
- *Positive:* Even if application code contains a bug or an attacker tries to inject another company's contact ID, PostgreSQL terminates the transaction with foreign key error `23503`. Cross-tenant data leaks are physically impossible.
- *Negative:* Parent tables must maintain a composite unique index on `(organization_id, id)`.

ADR-003: Append-Only Immutable Audit Log via Database Triggers

- **Status:** Accepted & Enforced
- **Context & Non-Technical Explanation:**

Under the UK Companies Act 2006 and international SOC 2 standards, financial audit trails must be non-repudiable. A company director or malicious insider must not be able to wipe audit records to hide fraudulent activities.

- **Decision:**

Create a PostgreSQL trigger function on `audit_logs` that unconditionally raises an exception on any `UPDATE` or `DELETE` statement.

- **Consequences & Trade-offs:**
- *Positive:* Guarantees statutory non-repudiation. Even if an attacker gains SQL superuser access through SQL injection, the database rejects attempts to alter audit records.
- *Negative:* Table grows monotonically; requires partition archiving strategies after 7+ years of operation.

ADR-004: Three-Tier Accounting Period Lockdown State Machine

- **Status:** Accepted & Enforced
- **Context & Non-Technical Explanation:**

Financial books move through stages of finality: daily entries during the month, reviews during month-end close, and final lockdown after HMRC tax submission. Soft warnings are insufficient because tired staff often dismiss warning dialogs.

- **Decision:**

Implement a strict three-state machine for accounting periods:

1. `OPEN`: Unrestricted posting for all roles.
2. `SOFT_LOCKED`: Closed for general staff; requires `ACCOUNTANT` or `OWNER` role to post adjustments.
3. `HARD_LOCKED`: Full immutable lockdown; rejects all postings and validation attempts.

- **Consequences & Trade-offs:**
- *Positive:* Protects historical tax returns and audited accounts from accidental corruption.
- *Negative:* Unlocking a hard-locked period requires an audited administrative procedure.

ADR-005: Modular Monolith Architecture over Microservices

- **Status:** Accepted & Enforced
- **Context & Non-Technical Explanation:**

Microservices split an application into dozens of tiny independent web services. While popular in large tech companies, microservices introduce network latency, complex distributed database transactions, and data inconsistency risks when posting multi-legged accounting journals.

- **Decision:**

Deploy blynt as a **Modular Monolith** using NestJS modules with clear domain boundaries, communicating in-process with shared relational transactions.

- **Consequences & Trade-offs:**
- *Positive:* ACID transactional integrity across all ledger postings. Instant rollback if any validation rule fails. Simple CI/CD deployment on a single container cluster.
- *Negative:* Requires disciplined module boundaries to prevent tight coupling.